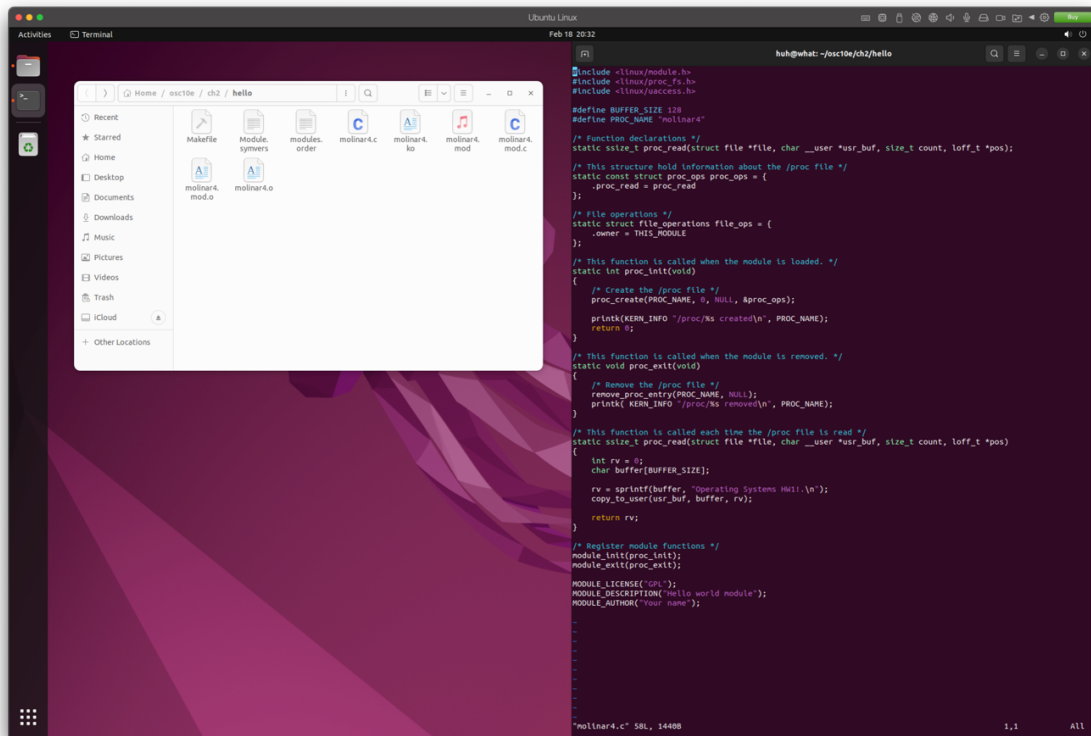


1. Programming Project III, Chapter 2: The /proc File System



```
huh@what: ~/osct0e/ch2/hello
#include <linux/module.h>
#include <linux/proc_fs.h>
#include <linux/jiffies.h>

#define BUFFER_SIZE 128
#define PROC_NAME "molinar4"

/* Function declarations */
static ssize_t proc_read(struct file *file, char __user *usr_buf, size_t count, loff_t *pos);

/* This structure hold information about the /proc file */
static const struct proc_ops proc_ops = {
    .proc_read = proc_read,
};

/* File operations */
static struct file_operations file_ops = {
    .owner = THIS_MODULE,
};

/* This function is called when the module is loaded. */
static int proc_init(void)
{
    /* Create the /proc file */
    proc_create(PROC_NAME, 0, NULL, &proc_ops);

    printk(KERN_INFO "/proc/%s created\n", PROC_NAME);
    return 0;
}

/* This function is called when the module is removed. */
static void proc_exit(void)
{
    /* Remove the /proc file */
    remove_proc_entry(PROC_NAME, NULL);
    printk(KERN_INFO "/proc/%s removed\n", PROC_NAME);
}

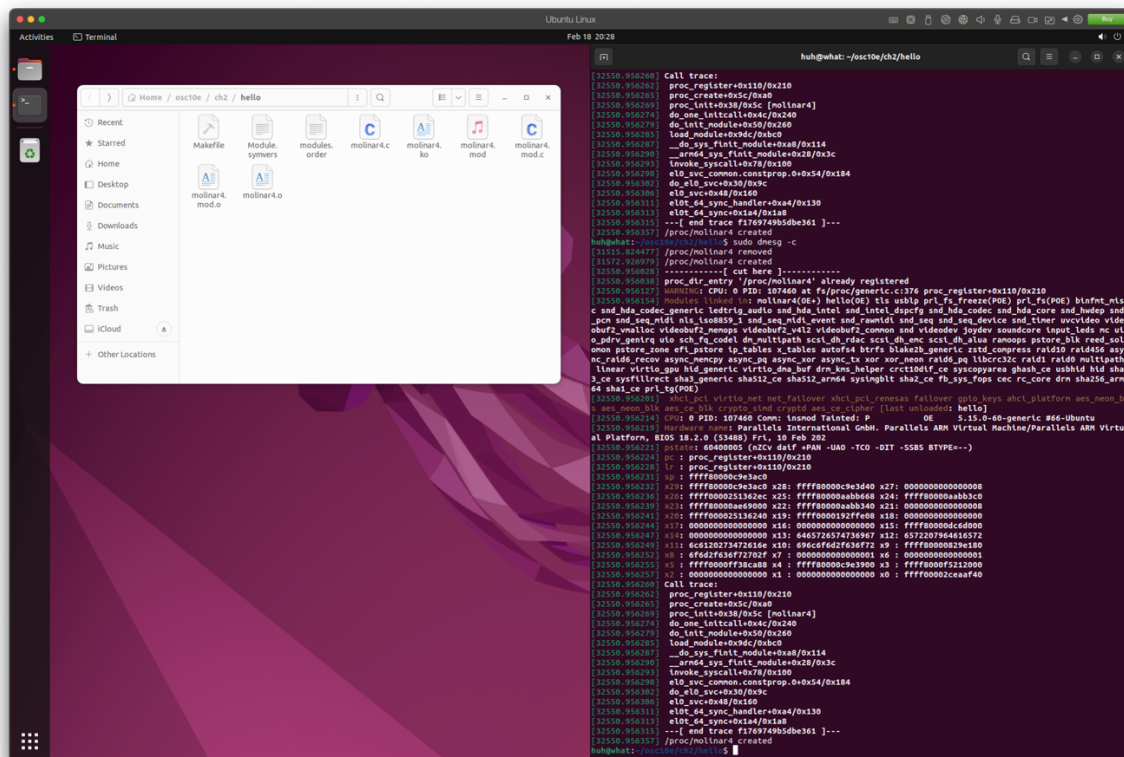
/* This function is called each time the /proc file is read */
static ssize_t proc_read(struct file *file, char __user *usr_buf, size_t count, loff_t *pos)
{
    int rv = 0;
    char buffer[BUFFER_SIZE];

    rv = sprintf(buffer, "Operating Systems HW1!\n");
    copy_to_user(usr_buf, buffer, rv);

    return rv;
}

/* Register module functions */
module_init(proc_init);
module_exit(proc_exit);

MODULE_LICENSE("GPL");
MODULE_DESCRIPTION("Hello world module");
MODULE_AUTHOR("your name");
```



```
huh@what: ~/osct0e/ch2/hello
[12550.956268] call trace:
[12550.956268] proc_register+0x110/0x210
[12550.956268] proc_create+0x5c/0xa0
[12550.956268] proc_init+0x18/0x5c [molinar4]
[12550.956274] do_one_initcall+0x4c/0x240
[12550.956279] do_init_module+0x50/0x260
[12550.956285] load_module+0x9dc/0xb30
[12550.956287] _do_sys_finit_module+0xa8/0x114
[12550.956290] __rmk4_sys_finit_module+0xc3c/0x3c
[12550.956295] invoke_syscall+0x78/0x100
[12550.956298] do_sys_svc+0x3b/0x9c
[12550.956300] eld_svc+0x48/0x60
[12550.956303] eld64_sys_handler+0x44/0x130
[12550.956311] eld64_syscall+0x14/0x18
[12550.956315] ---[ end trace f17697495db3e1 ]---
[12550.956317] /proc/molinar4 created
huh@what:~/osct0e/ch2/hello$ sudo dmesg -c
[1551.824077] /proc/molinar4 removed
[1552.916078] /proc/molinar4 created
[15550.956288] -----[ cut here ]-----
[12550.956288] proc_dfs_entry /proc/molinar4: Already registered
[12550.956288] molinar4: CPU: 0 PID: 107460 at fs/proc/generic.c:376 proc_register+0x110/0x210
[12550.956288] molinar4: molinar4(0x) hello(0x) tis usbp prl fs freeze(0x) prl fs(0x) blnfmt_mis
[12550.956288] c and hda codec generic ledtrig_audio and hda Intel snd Intel dspcfs and hda codec snd hda core snd hdaop snd
[12550.956288] _pcn snd_seq_nidi nls_iso8859_1 snd_seq_nidi event snd_rawmidi snd_seq_device snd_timer ucwvide vide
[12550.956288] vbi2 v4l2 videobuf2 vmemmgr videobuf2 v4l2 videobuf2_common and videobuf2_jpegdev soundcore tpm2 leds mc at
[12550.956288] o_pdrv_gentrg uio sch_fq_codel dm_multipath scsi_dh_rdc scsi_dh_anc scsi_dh_alua ramoops pstore_blk read_ao
[12550.956288] onon pstore_zone off_pstore ip_tables x_tables autofs4 btrfs blkzbd_generic zstd_compress raid10 raid456 asy
[12550.956288] nc_raid6 rcvcc asynq_mmcay asynq_pa asynq_sar asynq_tsr nor_nor_nor raid6_pq libcrch2c raid1 raid0 multipath
[12550.956288] linear virtio_gpu hid_generic virtio_dma_buf drn_hns_helper crc1t10d1f ce syscopyarea ghask ce usbbid hid sha
[12550.956288] s ce sysf1rect sha2_generic sha512_ce sha512_rnd4 sysngbit sha2_ce fb_sys_fops cec rc_core drm sha256_srn
[12550.956288] id sha2_ce prl fs(0x)
[12550.956288] ahci_pci virtio_net net_failover ahci_pci_remasa failover gpio_keys ahci_platform aes_xc
[12550.956288] aes_xc aes_xc aes_xc aes_xc aes_xc aes_xc aes_xc aes_xc aes_xc aes_xc aes_xc aes_xc aes_xc aes_xc
[12550.956288] CPU: 0 PID: 107460 Conn: Insnod Talated: P OE S:15.0-60-generic #66-Ubuntu
[12550.956288] al Platform, BIOS 18.2.0 (53488) Frl, 10 Feb 202
[12550.956288] psatit: 60400005 (n2cv dalf +PAN -UAO -TCD -DIT -SSBS BTYPE+...)
[12550.956288] pc : proc_register+0x110/0x210
[12550.956288] tr : proc_register+0x110/0x210
[12550.956288] x0 : ffffff0000c33ac0
[12550.956288] x1 : ffffff0000c33ac0 x2: ffffff0000c33ac0 x3: 0000000000000000
[12550.956288] x4: ffffff0000251362c x5: ffffff0000aabb468 x6: ffffff0000aabb468
[12550.956288] x7: ffffff0000000000 x8: ffffff0000aabb468 x9: 0000000000000000
[12550.956288] x10: ffffff000025136240 x11: ffffff0000192f0e8 x12: 0000000000000000
[12550.956288] x13: 0000000000000000 x14: 0000000000000000 x15: ffffff0000c33ac0
[12550.956288] x16: 0000000000000000 x17: 0000000000000000 x18: 0000000000000000
[12550.956288] x19: 0000000000000000 x20: 0000000000000000 x21: 65720784616172
[12550.956288] x22: 65720784616172 x23: 65720784616172 x24: 65720784616172
[12550.956288] x25: 65720784616172 x26: 65720784616172 x27: 65720784616172
[12550.956288] x28: 65720784616172 x29: 65720784616172 x30: 65720784616172
[12550.956288] x31: 65720784616172 x32: 65720784616172 x33: 65720784616172
[12550.956288] x34: 65720784616172 x35: 65720784616172 x36: 65720784616172
[12550.956288] x37: 65720784616172 x38: 65720784616172 x39: 65720784616172
[12550.956288] x40: 65720784616172 x41: 65720784616172 x42: 65720784616172
[12550.956288] x43: 65720784616172 x44: 65720784616172 x45: 65720784616172
[12550.956288] x46: 65720784616172 x47: 65720784616172 x48: 65720784616172
[12550.956288] x49: 65720784616172 x50: 65720784616172 x51: 65720784616172
[12550.956288] x52: 65720784616172 x53: 65720784616172 x54: 65720784616172
[12550.956288] x55: 65720784616172 x56: 65720784616172 x57: 65720784616172
[12550.956288] x58: 65720784616172 x59: 65720784616172 x60: 65720784616172
[12550.956288] x61: 65720784616172 x62: 65720784616172 x63: 65720784616172
[12550.956288] x64: 65720784616172 x65: 65720784616172 x66: 65720784616172
[12550.956288] x67: 65720784616172 x68: 65720784616172 x69: 65720784616172
[12550.956288] x70: 65720784616172 x71: 65720784616172 x72: 65720784616172
[12550.956288] x73: 65720784616172 x74: 65720784616172 x75: 65720784616172
[12550.956288] x76: 65720784616172 x77: 65720784616172 x78: 65720784616172
[12550.956288] x79: 65720784616172 x80: 65720784616172 x81: 65720784616172
[12550.956288] x82: 65720784616172 x83: 65720784616172 x84: 65720784616172
[12550.956288] x85: 65720784616172 x86: 65720784616172 x87: 65720784616172
[12550.956288] x88: 65720784616172 x89: 65720784616172 x90: 65720784616172
[12550.956288] x91: 65720784616172 x92: 65720784616172 x93: 65720784616172
[12550.956288] x94: 65720784616172 x95: 65720784616172 x96: 65720784616172
[12550.956288] x97: 65720784616172 x98: 65720784616172 x99: 65720784616172
[12550.956288] x100: 65720784616172 x101: 65720784616172 x102: 65720784616172
[12550.956288] x103: 65720784616172 x104: 65720784616172 x105: 65720784616172
[12550.956288] x106: 65720784616172 x107: 65720784616172 x108: 65720784616172
[12550.956288] x109: 65720784616172 x110: 65720784616172 x111: 65720784616172
[12550.956288] x112: 65720784616172 x113: 65720784616172 x114: 65720784616172
[12550.956288] x115: 65720784616172 x116: 65720784616172 x117: 65720784616172
[12550.956288] x118: 65720784616172 x119: 65720784616172 x120: 65720784616172
[12550.956288] x121: 65720784616172 x122: 65720784616172 x123: 65720784616172
[12550.956288] x124: 65720784616172 x125: 65720784616172 x126: 65720784616172
[12550.956288] x127: 65720784616172 x128: 65720784616172 x129: 65720784616172
[12550.956288] x130: 65720784616172 x131: 65720784616172 x132: 65720784616172
[12550.956288] x133: 65720784616172 x134: 65720784616172 x135: 65720784616172
[12550.956288] x136: 65720784616172 x137: 65720784616172 x138: 65720784616172
[12550.956288] x139: 65720784616172 x140: 65720784616172 x141: 65720784616172
[12550.956288] x142: 65720784616172 x143: 65720784616172 x144: 65720784616172
[12550.956288] x145: 65720784616172 x146: 65720784616172 x147: 65720784616172
[12550.956288] x148: 65720784616172 x149: 65720784616172 x150: 65720784616172
[12550.956288] x151: 65720784616172 x152: 65720784616172 x153: 65720784616172
[12550.956288] x154: 65720784616172 x155: 65720784616172 x156: 65720784616172
[12550.956288] x157: 65720784616172 x158: 65720784616172 x159: 65720784616172
[12550.956288] x160: 65720784616172 x161: 65720784616172 x162: 65720784616172
[12550.956288] x163: 65720784616172 x164: 65720784616172 x165: 65720784616172
[12550.956288] x166: 65720784616172 x167: 65720784616172 x168: 65720784616172
[12550.956288] x169: 65720784616172 x170: 65720784616172 x171: 65720784616172
[12550.956288] x172: 65720784616172 x173: 65720784616172 x174: 65720784616172
[12550.956288] x175: 65720784616172 x176: 65720784616172 x177: 65720784616172
[12550.956288] x178: 65720784616172 x179: 65720784616172 x180: 65720784616172
[12550.956288] x181: 65720784616172 x182: 65720784616172 x183: 65720784616172
[12550.956288] x184: 65720784616172 x185: 65720784616172 x186: 65720784616172
[12550.956288] x187: 65720784616172 x188: 65720784616172 x189: 65720784616172
[12550.956288] x190: 65720784616172 x191: 65720784616172 x192: 65720784616172
[12550.956288] x193: 65720784616172 x194: 65720784616172 x195: 65720784616172
[12550.956288] x196: 65720784616172 x197: 65720784616172 x198: 65720784616172
[12550.956288] x199: 65720784616172 x200: 65720784616172 x201: 65720784616172
[12550.956288] x202: 65720784616172 x203: 65720784616172 x204: 65720784616172
[12550.956288] x205: 65720784616172 x206: 65720784616172 x207: 65720784616172
[12550.956288] x208: 65720784616172 x209: 65720784616172 x210: 65720784616172
[12550.956288] x211: 65720784616172 x212: 65720784616172 x213: 65720784616172
[12550.956288] x214: 65720784616172 x215: 65720784616172 x216: 65720784616172
[12550.956288] x217: 65720784616172 x218: 65720784616172 x219: 65720784616172
[12550.956288] x220: 65720784616172 x221: 65720784616172 x222: 65720784616172
[12550.956288] x223: 65720784616172 x224: 65720784616172 x225: 65720784616172
[12550.956288] x226: 65720784616172 x227: 65720784616172 x228: 65720784616172
[12550.956288] x229: 65720784616172 x230: 65720784616172 x231: 65720784616172
[12550.956288] x232: 65720784616172 x233: 65720784616172 x234: 65720784616172
[12550.956288] x235: 65720784616172 x236: 65720784616172 x237: 65720784616172
[12550.956288] x238: 65720784616172 x239: 65720784616172 x240: 65720784616172
[12550.956288] x241: 65720784616172 x242: 65720784616172 x243: 65720784616172
[12550.956288] x244: 65720784616172 x245: 65720784616172 x246: 65720784616172
[12550.956288] x247: 65720784616172 x248: 65720784616172 x249: 65720784616172
[12550.956288] x250: 65720784616172 x251: 65720784616172 x252: 65720784616172
[12550.956288] x253: 65720784616172 x254: 65720784616172 x255: 65720784616172
[12550.956288] x256: 65720784616172 x257: 65720784616172 x258: 65720784616172
[12550.956288] x259: 65720784616172 x260: 65720784616172 x261: 65720784616172
[12550.956288] x262: 65720784616172 x263: 65720784616172 x264: 65720784616172
[12550.956288] x265: 65720784616172 x266: 65720784616172 x267: 65720784616172
[12550.956288] x268: 65720784616172 x269: 65720784616172 x270: 65720784616172
[12550.956288] x271: 65720784616172 x272: 65720784616172 x273: 65720784616172
[12550.956288] x274: 65720784616172 x275: 65720784616172 x276: 65720784616172
[12550.956288] x277: 65720784616172 x278: 65720784616172 x279: 65720784616172
[12550.956288] x280: 65720784616172 x281: 65720784616172 x282: 65720784616172
[12550.956288] x283: 65720784616172 x284: 65720784616172 x285: 65720784616172
[12550.956288] x286: 65720784616172 x287: 65720784616172 x288: 65720784616172
[12550.956288] x289: 65720784616172 x290: 65720784616172 x291: 65720784616172
[12550.956288] x292: 65720784616172 x293: 65720784616172 x294: 65720784616172
[12550.956288] x295: 65720784616172 x296: 65720784616172 x297: 65720784616172
[12550.956288] x298: 65720784616172 x299: 65720784616172 x300: 65720784616172
[12550.956288] x301: 65720784616172 x302: 65720784616172 x303: 65720784616172
[12550.956288] x304: 65720784616172 x305: 65720784616172 x306: 65720784616172
[12550.956288] x307: 65720784616172 x308: 65720784616172 x309: 65720784616172
[12550.956288] x310: 65720784616172 x311: 65720784616172 x312: 65720784616172
[12550.956288] x313: 65720784616172 x314: 65720784616172 x315: 65720784616172
[12550.956288] x316: 65720784616172 x317: 65720784616172 x318: 65720784616172
[12550.956288] x319: 65720784616172 x320: 65720784616172 x321: 65720784616172
[12550.956288] x322: 65720784616172 x323: 65720784616172 x324: 65720784616172
[12550.956288] x325: 65720784616172 x326: 65720784616172 x327: 65720784616172
[12550.956288] x328: 65720784616172 x329: 65720784616172 x330: 65720784616172
[12550.956288] x331: 65720784616172 x332: 65720784616172 x333: 65720784616172
[12550.956288] x334: 65720784616172 x335: 65720784616172 x336: 65720784616172
[12550.956288] x337: 65720784616172 x338: 65720784616172 x339: 65720784616172
[12550.956288] x340: 65720784616172 x341: 65720784616172 x342: 65720784616172
[12550.956288] x343: 65720784616172 x344: 65720784616172 x345: 65720784616172
[12550.956288] x346: 65720784616172 x347: 65720784616172 x348: 65720784616172
[12550.956288] x349: 65720784616172 x350: 65720784616172 x351: 65720784616172
[12550.956288] x352: 65720784616172 x353: 65720784616172 x354: 65720784616172
[12550.956288] x355: 65720784616172 x356: 65720784616172 x357: 65720784616172
[12550.956288] x358: 65720784616172 x359: 65720784616172 x360: 65720784616172
[12550.956288] x361: 65720784616172 x362: 65720784616172 x363: 65720784616172
[12550.956288] x364: 65720784616172 x365: 65720784616172 x366: 65720784616172
[12550.956288] x367: 65720784616172 x368: 65720784616172 x369: 65720784616172
[12550.956288] x370: 65720784616172 x371: 65720784616172 x372: 65720784616172
[12550.956288] x373: 65720784616172 x374: 65720784616172 x375: 65720784616172
[12550.956288] x376: 65720784616172 x377: 65720784616172 x378: 65720784616172
[12550.956288] x379: 65720784616172 x380: 65720784616172 x381: 65720784616172
[12550.956288] x382: 65720784616172 x383: 65720784616172 x384: 65720784616172
[12550.956288] x385: 65720784616172 x386: 65720784616172 x387: 65720784616172
[12550.956288] x388: 65720784616172 x389: 65720784616172 x390: 65720784616172
[12550.956288] x391: 65720784616172 x392: 65720784616172 x393: 65720784616172
[12550.956288] x394: 65720784616172 x395: 65720784616172 x396: 65720784616172
[12550.956288] x397: 65720784616172 x398: 65720784616172 x399: 65720784616172
[12550.956288] x400: 65720784616172 x401: 65720784616172 x402: 65720784616172
[12550.956288] x403: 65720784616172 x404: 65720784616172 x405: 65720784616172
[12550.956288] x406: 65720784616172 x407: 65720784616172 x408: 65720784616172
[12550.956288] x409: 65720784616172
```

```
/**
 * hello.c
 *
 * Kernel module that communicates with /proc file system.
 *
 * The makefile must be modified to compile this program.
 * Change the line "simple.o" to "hello.o"
 *
 * Operating System Concepts - 10th Edition
 * Copyright John Wiley & Sons - 2018
 */

#include <linux/init.h>
#include <linux/module.h>
#include <linux/kernel.h>
#include <linux/proc_fs.h>
#include <asm/uaccess.h>

#define BUFFER_SIZE 128

#define PROC_NAME "molinar4"
#define MESSAGE "Operating Systems HW1\n"

/**
 * Function prototypes
 */
ssize_t proc_read(struct file *file, char *buf, size_t count, loff_t *pos);

static const struct proc_ops proc_ops = {
    .read = proc_read
};

static struct file_operations file_ops = {
    .owner = THIS_MODULE,
```

```
};

/* This function is called when the module is loaded. */
int proc_init(void)
{
    // creates the /proc/hello entry
    // the following function call is a wrapper for
    // proc_create_data() passing NULL as the last argument
    proc_create(PROC_NAME, 0, NULL, &proc_ops);

    printk(KERN_INFO "/proc/%s created\n", PROC_NAME);

    return 0;
}

/* This function is called when the module is removed. */
void proc_exit(void)
{
    // removes the /proc/hello entry
    remove_proc_entry(PROC_NAME, NULL);

    printk(KERN_INFO "/proc/%s removed\n", PROC_NAME);
}

/**
 * This function is called each time the /proc/hello is read.
 *
 * This function is called repeatedly until it returns 0, so
 * there must be logic that ensures it ultimately returns 0
 * once it has collected the data that is to go into the
 * corresponding /proc file.
 *
 * params:
 *
```

```
* file:
* buf: buffer in user space
* count:
* pos:
*/
ssize_t proc_read(struct file *file, char __user *usr_buf, size_t count,
loff_t *pos)
{
    int rv = 0;
    char buffer[BUFFER_SIZE];
    static int completed = 0;

    if (completed)
    {
        completed = 0;
        return 0;
    }

    completed = 1;

    rv = sprintf(buffer, ".....\n");

    // copies the contents of buffer to userspace usr_buf
    copy_to_user(usr_buf, buffer, rv);

    return rv;
}

/* Macros for registering module entry and exit points. */
module_init(proc_init);
module_exit(proc_exit);

MODULE_LICENSE("GPL");
MODULE_DESCRIPTION("Process name change Module");
MODULE_AUTHOR("Richie");
```

2. Programming Problem 3.19: Two IPC Methods to measure command line argument time to execute:
- a. Shared Memory IPC

```
#include <stdio.h>
#include <stdlib.h>
#include <sys/mman.h>
#include <sys/time.h>
#include <sys/wait.h>
#include <fcntl.h>
#include <unistd.h>

// Define name and size of shared memory region
#define SHM_NAME "/my_shared_memory"
#define SHM_SIZE sizeof(struct timeval)

int main(int argc, char *argv[])
{
    // check if usr provided command as arg
    if (argc < 2)
    {
        fprintf(stderr, "usage: ./time <command>\n");
        exit(1);
    }

    struct timeval *start_time_ptr;
    // Open shared memory region with specified name and create
    if it doesn't exist
    int shm_fd = shm_open(SHM_NAME, O_CREAT | O_RDWR, 0666);
    if (shm_fd == -1)
    {
        perror("shm_open");
        exit(1);
    }

    // resize shared memory to size of struct timeval
    if (ftruncate(shm_fd, SHM_SIZE) == -1)
```

```
{
    perror("ftruncate");
    exit(1);
}

// Map shared memory region into program's virtual address
space
start_time_ptr = mmap(NULL, SHM_SIZE, PROT_READ | PROT_WRITE,
MAP_SHARED, shm_fd, 0);
if (start_time_ptr == MAP_FAILED)
{
    perror("mmap");
    exit(1);
}

pid_t pid = fork();
if (pid == -1)
{
    perror("fork");
    exit(1);
}
else if (pid == 0)
{ // child process
    // Record the starting time in shared memory
    gettimeofday(start_time_ptr, NULL);
    // exec specified command
    if (execvp(argv[1], &argv[1]) == -1)
    {
        perror("execvp");
        exit(1);
    }
}
else
{ // Parent Process
    int status;
    // Wait for child process to terminate
```

```
waitpid(pid, &status, 0);
if (WIFEXITED(status))
{
    struct timeval end_time;
    // Record ending time
    gettimeofday(&end_time, NULL);
    // Calculate elapsed time
    double elapsed_time = (end_time.tv_sec -
start_time_ptr->tv_sec) + (end_time.tv_usec - start_time_ptr-
>tv_usec) / 1000000.0;
    // Print elapsed time
    printf("Elapsed time: %f\n", elapsed_time);
}
else
{
    fprintf(stderr, "child process terminated
abnormally\n");
    exit(1);
}
// Clean up shared memory
if (munmap(start_time_ptr, SHM_SIZE) == -1)
{
    perror("munmap");
}
if (shm_unlink(SHM_NAME) == -1)
{
    perror("shm_unlink");
}
}

return 0;
}
```

```
ubuntu@ip-172-31-13-91: ~/operatingSystems -- ttys000 -- ㄸ%1
ubuntu@ip-172-31-13-91:~/operatingSystems$ vim ~/.vimrc
ubuntu@ip-172-31-13-91:~/operatingSystems$ vim ~/.vimrc
ubuntu@ip-172-31-13-91:~/operatingSystems$ gcc ./shm.c -o shm
./shm.c: In function 'main':
./shm.c:49:28: error: 'argv' undeclared (first use in this function); did you mean 'argvp'?
   49 |         if (execvp(argv[1], &argv[1]) == -1) {
      |                        ^~~~~
      |                        argvp
./shm.c:49:28: note: each undeclared identifier is reported only once for each function it appears in
ubuntu@ip-172-31-13-91:~/operatingSystems$ gcc ./shm.c -o shm
ubuntu@ip-172-31-13-91:~/operatingSystems$ ./shm ls
shm shm.c
Elapsed time: 0.001372
ubuntu@ip-172-31-13-91:~/operatingSystems$ ./shm pwd
/home/ubuntu/operatingSystems
Elapsed time: 0.002396
ubuntu@ip-172-31-13-91:~/operatingSystems$ ./shm ps -ef | grep "ubuntu"
root      841      766  0 01:19 ?        00:00:00 sshd: ubuntu [priv]
ubuntu    844        1  0 01:19 ?        00:00:00 /lib/systemd/systemd --user
ubuntu    845      844  0 01:19 ?        00:00:00 (sd-pam)
ubuntu    927      841  0 01:19 ?        00:00:00 sshd: ubuntu@pts/0
ubuntu    928      927  0 01:19 pts/0    00:00:00 -bash
root     1262      766  0 02:32 ?        00:00:00 sshd: ubuntu [priv]
ubuntu   1340     1262  0 02:32 ?        00:00:00 sshd: ubuntu@pts/1
ubuntu   1341     1340  0 02:32 pts/1    00:00:00 -bash
ubuntu   1956      928  0 03:31 pts/0    00:00:00 ./shm ps -ef
ubuntu   1957      928  0 03:31 pts/0    00:00:00 grep --color=auto ubuntu
ubuntu   1958     1956  0 03:31 pts/0    00:00:00 ps -ef
ubuntu@ip-172-31-13-91:~/operatingSystems$ ./shm ps
  PID TTY          TIME CMD
   928 pts/0    00:00:00 bash
  1959 pts/0    00:00:00 shm
  1960 pts/0    00:00:00 ps
Elapsed time: 0.005302
ubuntu@ip-172-31-13-91:~/operatingSystems$
```

b. Pipes IPC

```
#include <stdio.h>
#include <stdlib.h>
#include <sys/time.h>
#include <unistd.h>
#include <sys/wait.h>

int main(int argc, char *argv[])
{
    // check if user provided command as arg
    if (argc < 2)
    {
        fprintf(stderr, "Usage: ./time <command>\n");
        exit(1);
    }
}
```



```
int pipefd[2];
struct timeval start_time, end_time;
pid_t pid;

// create pipe for IPC
if (pipe(pipefd) == -1)
{
    perror("pipe");
    exit(1);
}

// Fork child process to execvp specified command
pid_t pid = fork();
if (pid == -1)
{
    perror("fork");
    exit(1);
}
else if (pid == 0)
{ // Child process
    // record starting time using gettimeofday()
    gettimeofday(&start_time, NULL);
    // write starting time to write end of pipe
    if (write(pipefd[1], &start_time, sizeof(struct timeval))
== -1)
    {
        perror("write");
        exit(1);
    }

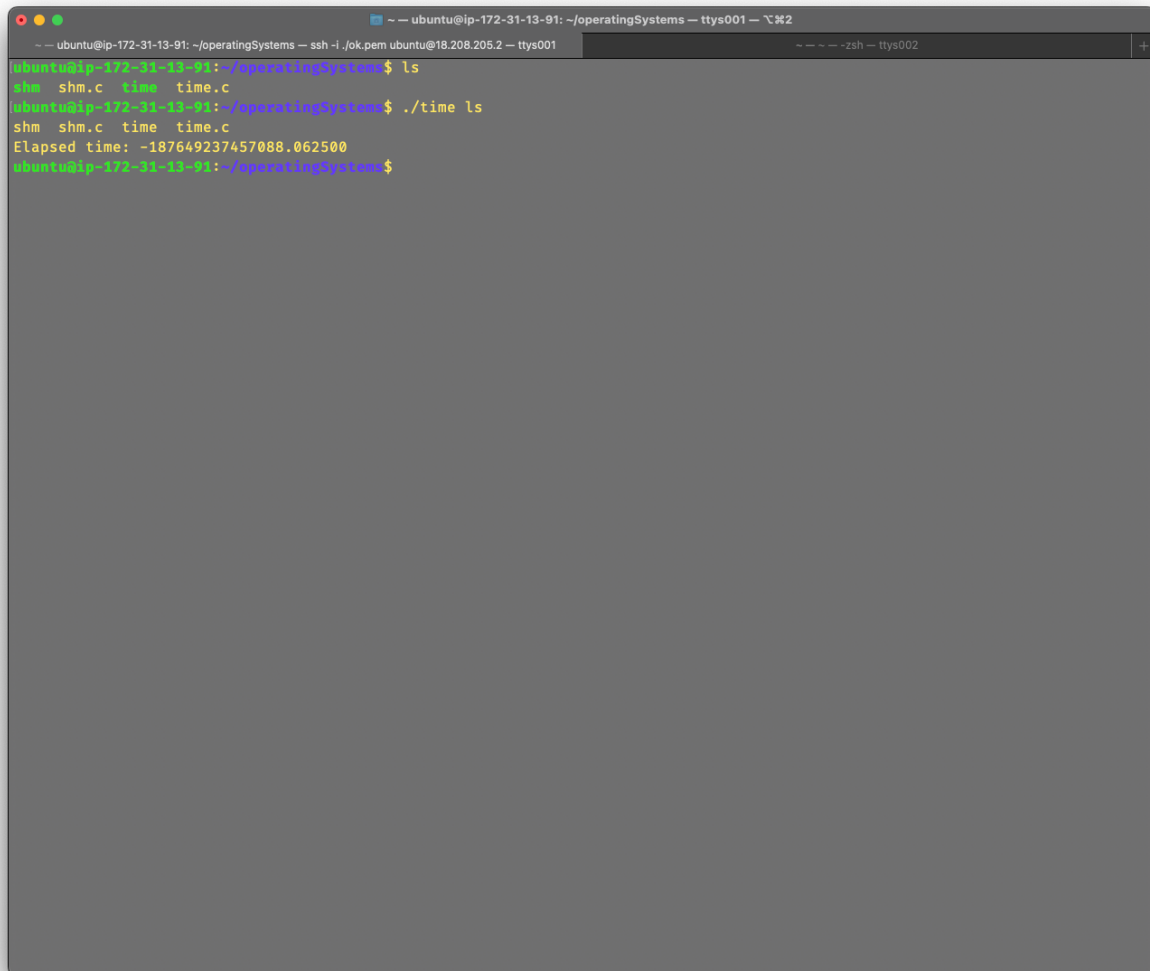
    // close read end of pipe since not needed in child
process
    if (close(pipefd[0]) == -1)
    {
        perror("close");
    }
}
```

```
        exit(1);
    }

    // execvp command line arg provided
    if (execvp(argv[1], &argv[1]) == -1)
    {
        perror("execvp");
        exit(1);
    }
}
else
{ // Parent Process
    int status;
    // wait for child process to terminate
    waitpid(pid, &status, 0);
    if (WIFEXITED(status))
    {
        // read ending time from read end of pipe
        if (read(pipefd[0], &end_time, sizeof(struct
timeval)) == -1)
        {
            perror("read");
            exit(1);
        }
        // calculate elapsed time and print it
        double elapsed_time = (end_time.tv_sec -
start_time.tv_sec) + (end_time.tv_usec - start_time.tv_usec) /
1000000.0;
        printf("Elapsed time: %f\n", elapsed_time);
    }
    else
    {
        // print error message if child process terminated
abnormally
        fprintf(stderr, "Child process terminated
abnormally\n");
    }
}
```

```
        exit(1);
    }
    if (close(pipefd[0]) == -1 || close(pipefd[1]) == -1)
    {
        // close both ends of pipe
        perror("close");
    }
}

return 0;
}
```



A terminal window screenshot showing the execution of a program. The window title is "ubuntu@ip-172-31-13-91: ~/operatingSystems - ssh -i ./.ok.pem ubuntu@18.208.205.2 - ttys001". The terminal output shows the user running 'ls' and then './time ls'. The output of './time ls' is 'shm shm.c time time.c' followed by 'Elapsed time: -187649237457088.062500'. The prompt returns to the user's shell.

```
ubuntu@ip-172-31-13-91: ~/operatingSystems$ ls
shm shm.c time time.c
ubuntu@ip-172-31-13-91: ~/operatingSystems$ ./time ls
shm shm.c time time.c
Elapsed time: -187649237457088.062500
ubuntu@ip-172-31-13-91: ~/operatingSystems$
```